Devoir 4

IFT714 : Traitement automatique des langues naturelles

Enseignant : Amine Trabelsi

Étudiant : Mamadou Senghor

CIP : senm1912

Université de Sherbrooke

Département d'informatique

Hiver 2026

Date de remise : 30 mars 2026

Table des matières

1 Question 1 2
1.1 (a) Mots de contexte 2
1.2 (b) Représentation vectorielle 2
1.3 (c) Distance euclidienne et similarité cosinus 2
1.3.1 Distance euclidienne 2
1.3.2 Similarité cosinus 3
1.3.3 Tableau récapitulatif 4

2 Question 2 4
2.1 (a) Tableau de similarité 4
2.1.1 Nombre total de mots communs 4
2.1.2 Produit scalaire 5
2.1.3 Similarité cosinus 5
2.1.4 Tableau récapitulatif 5
2.2 (b) Document le plus similaire selon le nombre total de mots communs . . 6
2.3 (c) Documents les plus similaires selon la similarité cosinus 6

3 Question 3 6
3.1 (a) Matrice de fréquence des bigrammes 6
3.2 (b) Probabilités de bigrammes non nulles 7
3.2.1 Probabilités de bigrammes non nulles 7
3.3 (c) Interprétation intuitive de la perplexité 8
3.4 (d) Calcul de la perplexité du corpus test 8

4 Question 4 9
4.1 Principe du lissage par interpolation linéaire 9
4.2 Calcul des différentes composantes 9
4.2.1 Probabilité trigramme 9
4.2.2 Probabilité bigramme 10
4.2.3 Probabilité unigramme 10
4.3 Probabilité interpolée finale 10

5 Question 5 10
5.1 (a) Comparaison entre `basicTokenize` et `nltkTokenize` 10
5.1.1 Top 10 avec `basicTokenize` 11
5.1.2 Top 10 avec `nltkTokenize` 11
5.1.3 Analyse des différences 11
5.2 (b) Fréquence des mots en fonction du rang 12
5.2.1 Analyse 12
5.3 (c) Modèle bigramme sans lissage 13
5.3.1 Analyse 13
5.4 (d) Modèle bigramme avec lissage add-α 13
5.4.1 Résultats 14
5.4.2 Analyse 14
5.4.3 Conclusion 15

1 Question 1

1.1 (a) Mots de contexte

On considère une fenêtre de taille 2, c'est-à-dire les deux mots à gauche et les deux mots à droite du mot cible. On ne conserve ensuite que les mots appartenant au vocabulaire de base

$$C = \{\text{while, pretty, purred, rubbing, big, furry, miaoued, barks, tiny}\}.$$

— Pour **kitten**, le mot apparaît dans deux phrases :
  — *... and the pretty kitten purred rubbing against her legs and then ...*
  — *... that the tiny kitten miaoued and she ...*
  Les mots de contexte appartenant à C sont donc :

$$\{\text{pretty, purred, rubbing, tiny, miaoued}\}.$$

— Pour **pussy**, dans la phrase *... the pretty furry pussy purred and miaoued ...*, les mots dans la fenêtre de taille 2 sont *pretty*, *furry*, *purred* et *and*. En ne gardant que les mots de C, on obtient :

$$\{\text{pretty, furry, purred}\}.$$

— Pour **dog**, dans la phrase *... then the big furry dog barks a while ...*, les mots dans la fenêtre de taille 2 sont *big*, *furry*, *barks* et *a*. En ne gardant que les mots de C, on obtient :

$$\{\text{big, furry, barks}\}.$$

1.2 (b) Représentation vectorielle

L'ordre des dimensions est :

$$(\text{while, pretty, purred, rubbing, big, furry, miaoued, barks, tiny}).$$

— Pour **kitten** :

$$\text{kitten} = (0, 1, 1, 1, 0, 0, 1, 0, 1)$$

— Pour **pussy** :

$$\text{pussy} = (0, 1, 1, 0, 0, 1, 0, 0, 0)$$

— Pour **dog** :

$$\text{dog} = (0, 0, 0, 0, 1, 1, 0, 1, 0)$$

1.3 (c) Distance euclidienne et similarité cosinus

On utilise les vecteurs suivants :

$$\text{kitten} = (0, 1, 1, 1, 0, 0, 1, 0, 1), \quad \text{pussy} = (0, 1, 1, 0, 0, 1, 0, 0, 0), \quad \text{dog} = (0, 0, 0, 0, 1, 1, 0, 1, 0)$$

1.3.1 Distance euclidienne

La distance euclidienne entre deux vecteurs x et y est donnée par :

$$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Entre kitten et pussy Les dimensions où les vecteurs diffèrent sont :

- rubbing (4e dimension) : $1 - 0 = 1$
- furry (6e dimension) : $0 - 1 = -1$
- miaoued (7e dimension) : $1 - 0 = 1$
- tiny (9e dimension) : $1 - 0 = 1$

$$d(\text{kitten}, \text{pussy}) = \sqrt{1^2 + (-1)^2 + 1^2 + 1^2} = \sqrt{4} = 2$$

Entre kitten et dog Les dimensions où les vecteurs diffèrent sont :

- pretty (2e dimension) : $1 - 0 = 1$
- purred (3e dimension) : $1 - 0 = 1$
- rubbing (4e dimension) : $1 - 0 = 1$
- big (5e dimension) : $0 - 1 = -1$
- furry (6e dimension) : $0 - 1 = -1$
- miaoued (7e dimension) : $1 - 0 = 1$
- barks (8e dimension) : $0 - 1 = -1$
- tiny (9e dimension) : $1 - 0 = 1$

$$d(\text{kitten}, \text{dog}) = \sqrt{1^2 + 1^2 + 1^2 + (-1)^2 + (-1)^2 + 1^2 + (-1)^2 + 1^2} = \sqrt{8} = 2\sqrt{2}$$

Entre pussy et dog Les dimensions où les vecteurs diffèrent sont :

- pretty (2e dimension) : $1 - 0 = 1$
- purred (3e dimension) : $1 - 0 = 1$
- big (5e dimension) : $0 - 1 = -1$
- barks (8e dimension) : $0 - 1 = -1$

$$d(\text{pussy}, \text{dog}) = \sqrt{1^2 + 1^2 + (-1)^2 + (-1)^2} = \sqrt{4} = 2$$

1.3.2 Similarité cosinus

La similarité cosinus entre deux vecteurs x et y est donnée par :

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$

On calcule d'abord les normes :

$$\|\text{kitten}\| = \sqrt{1^2 + 1^2 + 1^2 + 1^2 + 1^2} = \sqrt{5}$$

$$\|\text{pussy}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}$$

$$\|\text{dog}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}$$

Entre kitten et pussy

$$\text{kitten} \cdot \text{pussy} = 1 \times 1 + 1 \times 1 = 2$$

$$\cos(\text{kitten}, \text{pussy}) = \frac{2}{\sqrt{5}\sqrt{3}} = \frac{2}{\sqrt{15}}$$

Entre kitten et dog

$$\text{kitten} \cdot \text{dog} = 0$$

$$\cos(\text{kitten}, \text{dog}) = 0$$

Entre pussy et dog

$$\text{pussy} \cdot \text{dog} = 1 \times 1 = 1$$

$$\cos(\text{pussy}, \text{dog}) = \frac{1}{\sqrt{3}\sqrt{3}} = \frac{1}{3}$$

1.3.3 Tableau récapitulatif

Paires de mots	**Distance euclidienne**	**Similarité cosinus**
kitten et pussy	2	$\frac{2}{\sqrt{15}}$
kitten et dog	$2\sqrt{2}$	0
pussy et dog	2	$\frac{1}{3}$

Table 1 – Distances euclidiennes et similarités cosinus entre les mots cibles.

2 Question 2

On représente les trois documents par les vecteurs suivants, en respectant l'ordre

$$(\text{Ronaldo}, \text{Soccer}, \text{Messi}) :$$

$$A = (10, 50, 200), \qquad B = (400, 100, 10), \qquad C = (20, 10, 1)$$

2.1 (a) Tableau de similarité

2.1.1 Nombre total de mots communs

Pour deux documents, le nombre total de mots communs est obtenu en additionnant, pour chaque mot, le minimum des deux comptes.

Entre doc A et doc B

$$\min(10, 400) + \min(50, 100) + \min(200, 10) = 10 + 50 + 10 = 70$$

Entre doc B et doc C

$$\min(400, 20) + \min(100, 10) + \min(10, 1) = 20 + 10 + 1 = 31$$

Entre doc A et doc C

$$\min(10, 20) + \min(50, 10) + \min(200, 1) = 10 + 10 + 1 = 21$$

2.1.2 Produit scalaire

Entre doc A et doc B

$$A \cdot B = 10 \cdot 400 + 50 \cdot 100 + 200 \cdot 10 = 4000 + 5000 + 2000 = 11000$$

Entre doc B et doc C

$$B \cdot C = 400 \cdot 20 + 100 \cdot 10 + 10 \cdot 1 = 8000 + 1000 + 10 = 9010$$

Entre doc A et doc C

$$A \cdot C = 10 \cdot 20 + 50 \cdot 10 + 200 \cdot 1 = 200 + 500 + 200 = 900$$

2.1.3 Similarité cosinus

La similarité cosinus est donnée par :

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$

On calcule d'abord les normes :

$$\|A\| = \sqrt{10^2 + 50^2 + 200^2} = \sqrt{42600}$$

$$\|B\| = \sqrt{400^2 + 100^2 + 10^2} = \sqrt{170100}$$

$$\|C\| = \sqrt{20^2 + 10^2 + 1^2} = \sqrt{501}$$

Entre doc A et doc B

$$\cos(A, B) = \frac{11000}{\sqrt{42600}\sqrt{170100}} \approx 0.1292$$

Entre doc B et doc C

$$\cos(B, C) = \frac{9010}{\sqrt{170100}\sqrt{501}} \approx 0.9760$$

Entre doc A et doc C

$$\cos(A, C) = \frac{900}{\sqrt{42600}\sqrt{501}} \approx 0.1948$$

2.1.4 Tableau récapitulatif

Similarité	**Mots communs**	**Produit**	**Cosinus**
doc A & doc B	70	11000	≈ 0.1292
doc B & doc C	31	9010	≈ 0.9760
doc A & doc C	21	900	≈ 0.1948

Table 2 – Mesures de similarité entre les documents.

2.2 (b) Document le plus similaire selon le nombre total de mots communs

En utilisant le nombre total de mots communs, les deux documents les plus similaires sont **doc A** et **doc B**, car la valeur obtenue est la plus grande :

$$70 > 31 > 21$$

Ce résultat n'est pas totalement congruent avec le contenu des documents. En effet, doc B et doc C portent tous les deux sur Cristiano Ronaldo, alors que doc A porte sur Lionel Messi. Le critère du nombre total de mots communs ne tient pas compte de la proportion relative des mots dans chaque document. :contentReference[oaicite :0]index=0

2.3 (c) Documents les plus similaires selon la similarité cosinus

En utilisant la similarité cosinus, les deux documents les plus similaires sont **doc B** et **doc C**, car :

$$0.9760 > 0.1948 > 0.1292$$

Ce résultat est congruent avec le contenu des documents, puisque doc B et doc C concernent tous les deux Cristiano Ronaldo. :contentReference[oaicite :1]index=1

Dans ce contexte, le **produit scalaire** mesure une similarité brute dépendante de la taille des documents. Ainsi, deux documents longs peuvent avoir un produit scalaire élevé simplement parce qu'ils contiennent beaucoup de mots.

En revanche, la **similarité cosinus** normalise le produit scalaire par la longueur des vecteurs. Elle compare donc davantage la distribution relative des mots que leur quantité absolue. C'est pourquoi elle est plus adaptée ici pour juger de la proximité thématique réelle entre documents.

3 Question 3

Considérons le corpus suivant :

$$\langle s\rangle\ I\ enjoy\ tennis\ \langle /s\rangle$$

$$\langle s\rangle\ You\ enjoy\ soccer\ \langle /s\rangle$$

$$\langle s\rangle\ I\ dislike\ soccer\ tournaments\ \langle /s\rangle$$

$$\langle s\rangle\ You\ enjoy\ open\ tennis\ tournaments\ \langle /s\rangle$$

3.1 (a) Matrice de fréquence des bigrammes

Les bigrammes observés dans le corpus sont :

- $(\langle s\rangle, I)$: 2
- $(\langle s\rangle, you)$: 2
- $(I, enjoy)$: 1
- $(I, dislike)$: 1
- $(enjoy, tennis)$: 1
- $(enjoy, soccer)$: 1
- $(enjoy, open)$: 1

— $(tennis, \langle /s \rangle) : 1$
— $(tennis, tournaments) : 1$
— $(you, enjoy) : 2$
— $(soccer, \langle /s \rangle) : 1$
— $(soccer, tournaments) : 1$
— $(dislike, soccer) : 1$
— $(tournaments, \langle /s \rangle) : 2$
— $(open, tennis) : 1$
En utilisant l'ordre imposé

$$\langle s \rangle,\ I,\ enjoy,\ tennis,\ you,\ soccer,\ dislike,\ tournaments,\ open,\ \langle /s \rangle$$

pour les lignes et les colonnes, et en excluant $\langle /s \rangle$ comme ligne, on obtient la matrice suivante :

	$\langle s \rangle$	I	*enjoy*	*tennis*	*you*	*soccer*	*dislike*	*tournaments*	*open*	$\langle /s \rangle$
$\langle s \rangle$	0	2	0	0	2	0	0	0	0	0
I	0	0	1	0	0	0	1	0	0	0
enjoy	0	0	0	1	0	1	0	0	1	0
tennis	0	0	0	0	0	0	0	1	0	1
you	0	0	2	0	0	0	0	0	0	0
soccer	0	0	0	0	0	0	0	1	0	1
dislike	0	0	0	0	0	1	0	0	0	0
tournaments	0	0	0	0	0	0	0	0	0	2
open	0	0	0	1	0	0	0	0	0	0

TABLE 3 – Matrice de comptage des bigrammes.

3.2 (b) Probabilités de bigrammes non nulles

Dans un modèle bigramme sans lissage, la probabilité conditionnelle est donnée par :

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Les probabilités non nulles sont donc :

3.2.1 Probabilités de bigrammes non nulles

Les probabilités conditionnelles sont regroupées par mot précédent :

Mot précédent	Transitions et probabilités
$\langle s \rangle$	$P(I\|\langle s \rangle) = \frac{1}{2}, \quad P(you\|\langle s \rangle) = \frac{1}{2}$
I	$P(enjoy\|I) = \frac{1}{2}, \quad P(dislike\|I) = \frac{1}{2}$
$enjoy$	$P(tennis\|enjoy) = \frac{1}{3}, \quad P(soccer\|enjoy) = \frac{1}{3}, \quad P(open\|enjoy) = \frac{1}{3}$
$tennis$	$P(\langle /s \rangle\|tennis) = \frac{1}{2}, \quad P(tournaments\|tennis) = \frac{1}{2}$
you	$P(enjoy\|you) = 1$
$soccer$	$P(\langle /s \rangle\|soccer) = \frac{1}{2}, \quad P(tournaments\|soccer) = \frac{1}{2}$
$dislike$	$P(soccer\|dislike) = 1$
$tournaments$	$P(\langle /s \rangle\|tournaments) = 1$
$open$	$P(tennis\|open) = 1$

TABLE 4 – Probabilités de bigrammes non nulles.

3.3 (c) Interprétation intuitive de la perplexité

La perplexité d'un corpus test $W = w_1, \ldots, w_n$ est définie par :

$$PP(W) = P(W)^{-1/n} = 2^x \qquad \text{avec} \qquad x = -\frac{1}{n} \log_2 P(W)$$

Intuitivement, la perplexité mesure le degré d'incertitude du modèle face au corpus test. Elle peut être interprétée comme un nombre moyen de choix possibles à chaque position du texte. Plus la perplexité est faible, plus le modèle prédit bien le corpus. À l'inverse, une perplexité élevée indique que le modèle est plus incertain.

La perplexité varie en sens inverse de la probabilité du corpus :

— si $P(W)$ augmente, alors $PP(W)$ diminue ;

— si $P(W)$ diminue, alors $PP(W)$ augmente.

3.4 (d) Calcul de la perplexité du corpus test

Le corpus test est :

$$\langle s \rangle \ I \ enjoy \ tennis \ \langle /s \rangle$$

$$\langle s \rangle \ You \ enjoy \ open \ tennis \ tournaments \ \langle /s \rangle$$

D'après les probabilités calculées en (b), la probabilité de la première phrase est :

$$P_1 = P(I \mid \langle s \rangle) \cdot P(enjoy \mid I) \cdot P(tennis \mid enjoy) \cdot P(\langle /s \rangle \mid tennis)$$

$$P_1 = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{24}$$

La probabilité de la deuxième phrase est :

$$P_2 = P(you \mid \langle s \rangle) \cdot P(enjoy \mid you) \cdot P(open \mid enjoy) \cdot P(tennis \mid open) \cdot P(tournaments \mid tennis) \cdot P(\langle /s \rangle \mid t$$

$$P_2 = \frac{1}{2} \cdot 1 \cdot \frac{1}{3} \cdot 1 \cdot \frac{1}{2} \cdot 1 = \frac{1}{12}$$

La probabilité totale du corpus test est donc :

$$P(W) = P_1 \cdot P_2 = \frac{1}{24} \cdot \frac{1}{12} = \frac{1}{288}$$

La longueur n du corpus test exclut les tokens $\langle s \rangle$ mais inclut les tokens $\langle /s \rangle$:

$$n = 4 + 6 = 10$$

Ainsi, la perplexité vaut :

$$PP(W) = P(W)^{-1/n} = \left(\frac{1}{288}\right)^{-1/10} = 288^{1/10} \approx 1.7617$$

Donc,

$$\boxed{PP(W) \approx 1.7617}$$

4 Question 4

On veut estimer la probabilité de trigramme

$$P(\text{Marilyn} \mid I\ am)$$

en utilisant un lissage par interpolation linéaire simple avec des poids égaux λ. Le corpus donné est :

$$\langle s \rangle\ I\ am\ Marilyn\ \langle /s \rangle$$

$$\langle s \rangle\ Marilyn\ I\ am\ \langle /s \rangle$$

$$\langle s \rangle\ I\ do\ like\ going\ out\ for\ dinner\ tonight\ \langle /s \rangle$$

4.1 Principe du lissage par interpolation linéaire

Avec des poids égaux, on prend :

$$\lambda_1 = \lambda_2 = \lambda_3 = \frac{1}{3}$$

Ainsi, la probabilité interpolée s'écrit :

$$P(\text{Marilyn} \mid I\ am) = \frac{1}{3}P_{\text{tri}}(\text{Marilyn} \mid I\ am) + \frac{1}{3}P_{\text{bi}}(\text{Marilyn} \mid am) + \frac{1}{3}P_{\text{uni}}(\text{Marilyn})$$

4.2 Calcul des différentes composantes

4.2.1 Probabilité trigramme

Le contexte $I\ am$ apparaît deux fois dans le corpus :
— une fois suivi de Marilyn ;
— une fois suivi de $\langle /s \rangle$.
Donc :

$$C(I\ am,\ Marilyn) = 1 \qquad \text{et} \qquad C(I\ am) = 2$$

Par conséquent :

$$P_{\text{tri}}(\text{Marilyn} \mid I\ am) = \frac{1}{2}$$

4.2.2 Probabilité bigramme

Le mot *am* apparaît deux fois comme contexte :
— une fois suivi de Marilyn ;
— une fois suivi de $\langle /s \rangle$.
Donc :

$$C(am,\ Marilyn) = 1 \qquad \text{et} \qquad C(am) = 2$$

Par conséquent :

$$P_{\text{bi}}(\text{Marilyn} \mid am) = \frac{1}{2}$$

4.2.3 Probabilité unigramme

Le mot Marilyn apparaît deux fois dans le corpus :

$$C(\text{Marilyn}) = 2$$

En comptant tous les tokens du corpus, y compris $\langle s \rangle$ et $\langle /s \rangle$, on obtient :

$$N = 5 + 5 + 10 = 20$$

Ainsi :

$$P_{\text{uni}}(\text{Marilyn}) = \frac{2}{20} = \frac{1}{10}$$

4.3 Probabilité interpolée finale

On remplace dans la formule :

$$P(\text{Marilyn} \mid I\ am) = \frac{1}{3} \cdot \frac{1}{2} + \frac{1}{3} \cdot \frac{1}{2} + \frac{1}{3} \cdot \frac{1}{10}$$

$$= \frac{1}{6} + \frac{1}{6} + \frac{1}{30} = \frac{5}{30} + \frac{5}{30} + \frac{1}{30} = \frac{11}{30}$$

Donc :

$$\boxed{P(\text{Marilyn} \mid I\ am) = \frac{11}{30} \approx 0.3667}$$

5 Question 5

5.1 (a) Comparaison entre `basicTokenize` et `nltkTokenize`

Nous avons extrait les 10 mots les plus fréquents du corpus d'entraînement en utilisant deux méthodes de tokenisation : `basicTokenize` et `nltkTokenize`.

5.1.1 Top 10 avec `basicTokenize`

Rang	Mot	Fréquence
1	unk	61019
2	the	45302
3	of	25379
4	and	18067
5	to	16515
6	a	14371
7	in	14231
8	is	7466
9	that	6484
10	for	6434

TABLE 5 – Top 10 des mots avec `basicTokenize`.

5.1.2 Top 10 avec `nltkTokenize`

Rang	Mot	Fréquence
1	unk	61019
2	the	45885
3	of	25427
4	,	23570
5	and	18346
6	.	17532
7	to	16606
8	a	14721
9	in	14358
10	is	7702

TABLE 6 – Top 10 des mots avec `nltkTokenize`.

5.1.3 Analyse des différences

On observe plusieurs différences importantes entre les deux tokenisations :

— **Présence de la ponctuation :** avec `nltkTokenize`, les signes de ponctuation tels que la virgule (,) et le point (.) apparaissent parmi les mots les plus fréquents. Cela est dû au fait que ce tokenizer sépare explicitement la ponctuation en tokens distincts.

— **Absence de ponctuation dans `basicTokenize` :** la méthode `basicTokenize` ne sépare pas la ponctuation des mots, ce qui empêche ces symboles d'apparaître comme tokens fréquents.

— **Différences de fréquence :** certaines fréquences varient légèrement entre les deux méthodes, car la segmentation du texte n'est pas identique.

— **Impact du token `unk` :** le token `unk` est le plus fréquent dans les deux cas, ce qui indique la présence de nombreux mots inconnus ou rares remplacés dans le corpus.

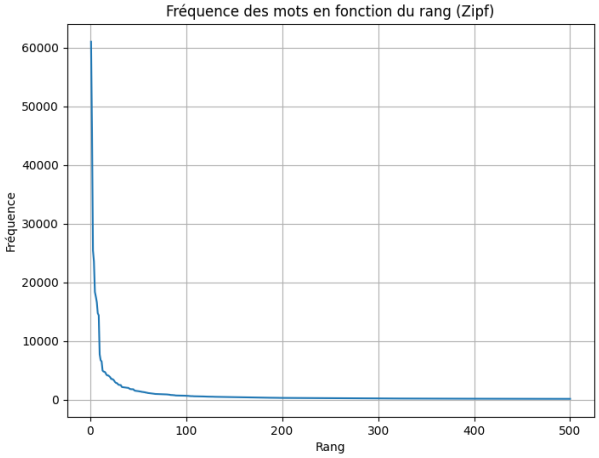

FIGURE 1 – Fréquence des mots en fonction du rang (échelle linéaire).

Ces différences montrent que le choix du tokenizer influence directement la distribution des mots, ce qui peut avoir un impact significatif sur les modèles de langage entraînés sur ces données.

5.2 (b) Fréquence des mots en fonction du rang

Nous avons tracé la fréquence des mots du corpus d'entraînement en fonction de leur rang (du mot le plus fréquent au moins fréquent), en utilisant la tokenisation `nltkTokenize`.

5.2.1 Analyse

On observe une forte décroissance de la fréquence des mots en fonction du rang. Les premiers mots sont extrêmement fréquents, tandis que la majorité des mots apparaissent très rarement.

Cette distribution suit la loi de Zipf, qui stipule que la fréquence d'un mot est inversement proportionnelle à son rang :

$$f(r) \propto \frac{1}{r}$$

Ainsi, quelques mots très fréquents dominent le corpus (par exemple les mots fonctionnels comme `the`, `,`, `.`, `of`, `and`), tandis qu'un grand nombre de mots rares apparaissent très peu souvent.

Cette propriété a des implications importantes en traitement automatique des langues, notamment en ce qui concerne la **sparsité** des données et la nécessité d'utiliser des techniques de lissage dans les modèles de langage. En effet, la présence de nombreux mots rares rend difficile l'estimation fiable des probabilités des événements peu fréquents ou non observés.

Pour mieux visualiser la loi de Zipf, on utilise généralement une échelle logarithmique sur les deux axes, où la relation devient approximativement linéaire.

En conclusion, ce comportement confirme que la distribution des mots dans le langage naturel est fortement déséquilibrée, ce qui doit être pris en compte lors de la conception des modèles de langage.

5.3 (c) Modèle bigramme sans lissage

Nous avons entraîné un modèle de langage bigramme ($n = 2$) sans lissage en utilisant la tokenisation `basicTokenize` et l'ensemble complet du corpus d'entraînement.

Les perplexités obtenues sont les suivantes :

— Perplexité sur l'ensemble d'entraînement : 69.88

— Perplexité sur l'ensemble de validation : ∞

5.3.1 Analyse

La perplexité sur l'ensemble d'entraînement est relativement faible, ce qui indique que le modèle s'ajuste bien aux données qu'il a observées.

En revanche, la perplexité sur l'ensemble de validation est infinie. Cela est dû au fait que certains bigrammes présents dans le corpus de validation n'ont jamais été observés dans le corpus d'entraînement. Dans ce cas, la probabilité associée à ces bigrammes est nulle :

$$P(w_i \mid w_{i-1}) = 0$$

Le calcul de la perplexité implique le logarithme de ces probabilités. Or, $\log(0)$ n'est pas défini, ce qui entraîne une valeur infinie pour la perplexité. Plus précisément, cela implique que :

$$P(W) = 0 \quad \Rightarrow \quad \log P(W) = -\infty$$

Lors de l'exécution, Python génère un avertissement `RuntimeWarning: divide by zero encountered in log`, confirmant la présence de ces probabilités nulles.

Ce résultat met en évidence une limitation importante des modèles de langage sans lissage : ils ne peuvent pas gérer les événements non observés et ne généralisent pas bien aux nouvelles données.

En pratique, un tel modèle est donc inutilisable sur des données réelles, car il attribue une probabilité nulle à toute séquence contenant un événement non observé.

Cela justifie l'utilisation de techniques de lissage, telles que le lissage additif (add-α), afin d'attribuer une probabilité non nulle à tous les événements possibles.

5.4 (d) Modèle bigramme avec lissage add-α

Nous avons entraîné un modèle de langage bigramme ($n = 2$) avec lissage additif (add-α) en utilisant la tokenisation `basicTokenize`. Le lissage additif attribue une probabilité non nulle à tous les bigrammes selon la formule :

$$P(w_j \mid w_i) = \frac{C(w_i, w_j) + \alpha}{C(w_i) + \alpha|V|}$$

où $|V|$ est la taille du vocabulaire et $C(w_i) = \sum_j C(w_i, w_j)$.

Différentes valeurs de α ont été testées :

$$\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 1.5, 2\}$$

5.4.1 Résultats

α	Perplexité train	Perplexité validation
10^{-5}	70.37	4542.59
10^{-4}	74.15	2219.99
10^{-3}	95.93	1333.48
10^{-2}	184.99	**1164.73**
10^{-1}	540.16	1545.30
1	2003.89	2993.01
1.5	2504.84	3453.55
2	2918.40	3831.11

TABLE 7 – Perplexité en fonction de α.

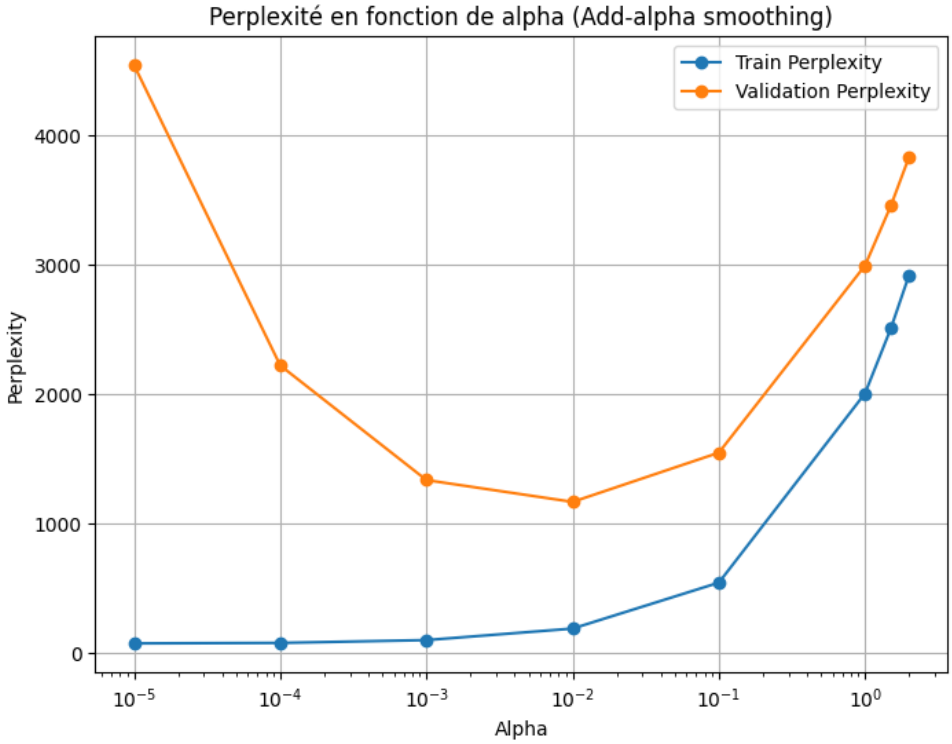

FIGURE 2 – Perplexité en fonction de α pour les ensembles d'entraînement et de validation.

5.4.2 Analyse

On observe que la perplexité sur l'ensemble de validation diminue fortement lorsque α augmente à partir de très petites valeurs, puis atteint un minimum pour $\alpha = 10^{-2}$, avant d'augmenter à nouveau pour des valeurs plus grandes.

Ce comportement illustre le compromis entre sous-apprentissage et sur-apprentissage :

— **Petit α :** le modèle est proche du modèle sans lissage et reste très sensible aux événements non observés, ce qui entraîne une forte perplexité sur la validation.

— **α intermédiaire :** le modèle généralise mieux tout en conservant une structure probabiliste informative. Cela correspond à la meilleure performance observée.

— **Grand α :** le modèle devient trop lissé et tend vers une distribution plus uniforme, ce qui dégrade ses performances.

On observe également que la perplexité sur l'ensemble d'entraînement augmente avec α, ce qui est attendu puisque le lissage réduit l'adaptation du modèle aux données d'entraînement. De plus, la perplexité sur l'entraînement reste toujours inférieure à celle sur la validation, ce qui est normal car le modèle est optimisé sur les données d'entraînement.

5.4.3 Conclusion

Le lissage additif permet de résoudre le problème des probabilités nulles et d'obtenir une perplexité finie sur l'ensemble de validation. Il améliore également la stabilité numérique du modèle en évitant les probabilités nulles.

La recherche de la meilleure valeur de α sur l'ensemble de validation illustre le principe de la validation croisée : on utilise un ensemble distinct pour régler les hyperparamètres, afin d'éviter de sur-ajuster aux données d'entraînement.

Le choix de α est crucial : une valeur trop faible ou trop élevée dégrade les performances, tandis qu'une valeur intermédiaire ($\alpha = 10^{-2}$ dans notre cas) permet d'obtenir un bon compromis entre précision et généralisation.